

# Enumerações (enum) e typedef

Dar nome aos valores e aos tipos — aplicado a registros

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

[andreyoshiaki@dcomp.ufs.br](mailto:andreyoshiaki@dcomp.ufs.br)

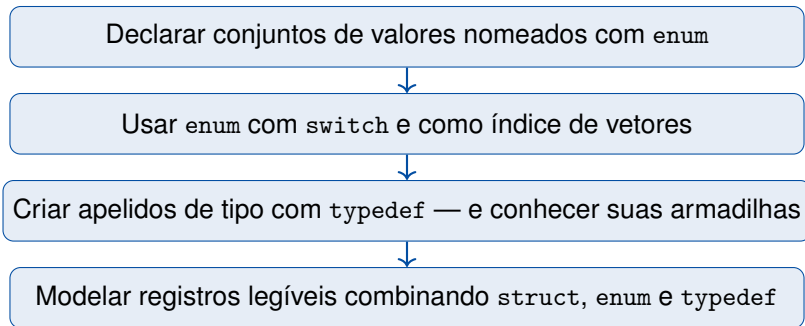
Um programa para começar

Enumerações

`typedef`

Registros que se explicam sozinhos

Tarefas



**Um programa para começar**

## Como estava na aula passada

```
1 struct Aluno {  
2     char nome[20];  
3     int matricula;  
4     double media;  
5     int situacao; /* 0, 1 ou 2 */  
6 };  
7  
8 a.situacao = 2;  
9 if (a.situacao == 1) { ... }
```

## As perguntas que sobram

- 2 significa aprovado ou reprovado?
- Onde está escrito que só valem 0, 1 e 2?
- Quem lê `if (a.situacao == 1)` seis meses depois entende?
- E se alguém escrever `a.situacao = 9`?

## Números mágicos

Valores literais sem nome: compilam perfeitamente e escondem a intenção.

```
1  #include <stdio.h>
2
3  typedef enum { REPROVADO, RECUPERACAO, APROVADO } Situacao;
4
5  typedef struct {
6      char    nome[20];
7      int     matricula;
8      double  media;
9      Situacao situacao;
10 } Aluno;
11
12 Situacao classifica(double media) {
13     if (media >= 7.0) return APROVADO;
14     if (media >= 5.0) return RECUPERACAO;
15     return REPROVADO;
16 }
```

## ... e o programa principal



```
18 void imprime_situacao(Situacao s) {
19     switch (s) {
20         case APROVADO:    printf("Aprovado");    break;
21         case RECUPERACAO: printf("Recuperacao"); break;
22         case REPROVADO:   printf("Reprovado");   break;
23     }
24 }
25
26 int main(void) {
27     Aluno a = {"Ana Souza", 202600123, 8.75, REPROVADO};
28     a.situacao = classifica(a.media);
29     printf("%s (%d) media %.2f\n", a.nome, a.matricula, a.media);
30     printf("situacao: %d = ", a.situacao);
31     imprime_situacao(a.situacao);
32     printf("\nsizeof(Situacao) = %zu\n", sizeof(Situacao));
33     return 0;
34 }
```

## Em duplas, anotem a previsão (3–4 min) — sem computador!

1. Quais são exatamente as três linhas impressas na tela?
2. `printf("%d", a.situacao)` imprime o quê? Por quê esse número?
3. Quantos *bytes* ocupa `Situacao`? E onde está escrito isso?
4. A linha 27 inicializa `situacao` como `REPROVADO` e a linha 28 sobrescreve. Faz diferença o valor inicial?

## Regra do jogo

Escreva um número, não um “mais ou menos”. A pergunta 2 é a porta de entrada da aula: o texto `APROVADO` vira *alguma coisa* antes de o programa rodar.

```
$ gcc -Wall ficha_situacao.c -o ficha_situacao
$ ./ficha_situacao
Ana Souza (202600123) media 8.75
situacao: 2 = Aprovado
sizeof(Situacao) = 4
```

## As perguntas que guiam o resto da aula

- De onde saiu o **2**? Ninguém escreveu 2 no programa.
- Situacao ocupa 4 bytes: então é um tipo inteiro?
- Aluno e Situacao aparecem sem struct nem enum. Quem permitiu?

# Enumerações

## Definição

Uma **enumeração** (`enum`) é um tipo inteiro cujos valores válidos recebem nomes no próprio código. Cada nome é uma *constante de enumeração* — um valor `int` conhecido em tempo de compilação.

## Intuição

Você não inventa os valores: você *lista* os que existem. “Situação de aluno” tem exatamente três valores possíveis — o `enum` escreve essa lista no programa, em vez de deixá-la na sua cabeça.

```
enum Situacao { REPROVADO, RECUPERACAO, APROVADO };  
/*   ^tag           ^----- constantes de enumeracao -----^ */  
  
enum Situacao s;           /* declara variavel: precisa da palavra enum */  
s = APROVADO;             /* as constantes sao usadas soltas, sem tag */
```

- A **tag** (Situacao) nomeia o tipo; as constantes vivem fora dela, direto no escopo do arquivo.
- Por isso APROVADO se escreve sozinho — e por isso dois enum não podem repetir o mesmo nome de constante.
- Convenção usual: constantes em MAIUSCULAS, tag em PascalCase.

# Os valores são inteiros consecutivos a partir de zero



*o compilador atribui  
os valores na ordem  
em que você escreveu*

## Consequência prática

Trocar a ordem dos nomes na declaração troca os números. Isso importa se os valores forem gravados em arquivo ou trocados com outro programa.

```
1  /* começar em 1 */
2  enum Mes { JAN = 1, FEV, MAR, ABR };
3  /* JAN=1 FEV=2 MAR=3 ABR=4 */
4
5  /* valores independentes */
6  enum Permissao {
7      LEITURA = 1,
8      ESCRITA = 2,
9      EXECUCAO = 4
10 };
```

- Após um valor explícito, os seguintes continuam a contagem (+1 a cada nome).
- Potências de dois permitem combinar com |: LEITURA | ESCRITA vale 3.
- Dois nomes podem ter o mesmo valor — é legal.

## Quando fixar valores

Quando o número tem significado externo (código de erro, byte de protocolo, mês). Caso contrário, deixe o compilador contar.

```
1 void imprime_situacao(Situacao s) {  
2     switch (s) {  
3         case APROVADO: printf("Aprovado"); break;  
4         case REPROVADO: printf("Reprovado"); break;  
5     } /* faltou RECUPERACAO */  
6 }
```

```
$ gcc -Wall sw.c -o sw  
sw.c:4:13: warning: enumeration value 'RECUPERACAO' not handled in switch [-Wswitch]
```

## Por que isso é valioso

Com `int situacao` o compilador nada teria a reclamar. Com `enum`, ele conhece a lista inteira e cobra o caso esquecido — o erro que aparece meses depois, ao acrescentar um valor novo. Atenção: um `default`: desliga esse aviso.

```
1 Situacao s = 7;           /* compila, sem aviso, e imprime 7 */
2 s = APROVADO + 1;        /* 3: um valor que nao tem nome */
3 int n = APROVADO;        /* legal: enum vira int sozinho */
```

## O que C garante — e o que não garante

C garante os *nomes*; não garante que a variável só receba valores da lista. Uma enumeração é documentação executável, não uma trava.

## Como se proteger

Valide o que vem de fora antes de atribuir — `if (v >= REPROVADO && v <= APROVADO)` — e não use aritmética para “andar até o próximo estado” sem checar o limite.

|                              | #define         | const int  | enum       |
|------------------------------|-----------------|------------|------------|
| Quem processa                | pré-processador | compilador | compilador |
| Aparece no depurador         | não             | sim        | sim        |
| Agrupar valores relacionados | não             | não        | <b>sim</b> |
| Serve de case em switch      | sim             | não        | <b>sim</b> |
| Tamanho de vetor (C89)       | sim             | não        | <b>sim</b> |
| Aviso de case faltante       | não             | não        | <b>sim</b> |

## Regra prática

Para um conjunto fechado de valores relacionados: `enum`. Para uma constante isolada (TAM, PI): `#define` OU `const`.

# O truque do contador: `enum` como índice

```
1 typedef enum {
2     REPROVADO, RECUPERACAO, APROVADO,
3     NUM_SITUACOES      /* sentinela = 3 */
4 } Situacao;
5
6 int cont[NUM_SITUACOES] = {0};
7
8 for (int i = 0; i < n; i++)
9     cont[turma[i].situacao]++;
10
11 printf("%d aprovados\n", cont[APROVADO]);
```

|             |   |          |
|-------------|---|----------|
| REPROVADO   |   |          |
| RECUPERACAO |   | APROVADO |
| 2           | 1 | 3        |
| 0           | 1 | 2        |

*o índice do vetor é  
a própria situação*

## Por que a sentinela funciona

NUM\_SITUACOES é o próximo número da contagem: a *quantidade* de valores.

## O que aprendemos

- `enum` dá nome a um conjunto fechado de valores inteiros.
- Sem valores explícitos, a contagem começa em 0 e segue a ordem escrita.
- Combina com `switch`: o compilador cobra os casos faltantes.
- Serve de índice de vetor; a sentinela final dá o número de valores.
- Não é uma trava: a variável ainda aceita qualquer inteiro.

## Ainda em aberto

Escrevemos `Situacao s` sem a palavra `enum` na frente. Falta explicar quem autorizou — é o próximo tópico.

typedef

## Sem typedef

```
1 struct Aluno a;  
2 struct Aluno turma[40];  
3 enum Situacao s;  
4  
5 enum Situacao classifica(double m);  
6 void relatorio(struct Aluno t[], int n);
```

## Com typedef

```
1 Aluno a;  
2 Aluno turma[40];  
3 Situacao s;  
4  
5 Situacao classifica(double m);  
6 void relatorio(Aluno t[], int n);
```

## O ganho não é só digitar menos

Aluno e Situacao passam a ser nomes de tipo como int ou double. A assinatura da função fica na linguagem do problema, não na da implementação.

typedef cria um **apelido** (sinônimo) para um tipo já existente. Não cria tipo novo, não aloca memória, não muda nada em tempo de execução — é uma declaração para o compilador.

## A regra para ler qualquer typedef

Escreva a declaração como se fosse uma *variável* daquele tipo; ponha `typedef` na frente. O nome que seria da variável vira o nome do tipo.

unsigned long matricula; → typedef unsigned long Matricula;

variável                      tipo

## 1. Só a tag

```
1 struct Aluno {  
2     char nome[20];  
3     double media;  
4 };  
5  
6 struct Aluno a;
```

## 2. Só o apelido

```
1 typedef struct {  
2     char nome[20];  
3     double media;  
4 } Aluno;  
5  
6 Aluno a;
```

## 3. Tag + apelido

```
1 typedef struct Aluno {  
2     char nome[20];  
3     double media;  
4 } Aluno;  
5  
6 Aluno a;  
7 struct Aluno b;
```

### As três geram o mesmo tipo em memória

A forma 2 é a mais comum. A forma 3 é necessária quando o registro precisa se referir a si mesmo (um campo apontando para outro registro do mesmo tipo) — caso que veremos na aula de ponteiros e listas.

```
1 typedef enum {  
2     REPROVADO, RECUPERACAO, APROVADO  
3 } Situacao;  
4  
5 Situacao s = APROVADO;
```

```
1 typedef unsigned long Matricula;  
2 typedef double Nota;  
3  
4 Matricula m = 202600123;  
5 Nota      n = 8.75;
```

## Por que apelidar um tipo básico

- **Intenção:** Nota media; diz mais que double media;.
- **Portabilidade:** se amanhã a matrícula precisar de mais dígitos, muda-se *uma* linha e o programa inteiro acompanha.
- É essa a lógica de size\_t, time\_t e FILE na biblioteca padrão.

```
1  typedef double Nota;  
2  typedef double Salario;  
3  
4  Nota    n = 8.75;  
5  Salario s = 3200.0;  
6  
7  n = s;           /* compila: os dois SAO double */
```

## Apelido, não tipo distinto

O compilador não distingue Nota de Salario — ambos são double. typedef organiza a leitura; não impede misturas sem sentido. Já um struct novo é um tipo distinto, e esses nunca se misturam por acidente.

```
1  typedef char Nome[20];           /* Nome E um vetor de 20 chars */
2
3  Nome n = "Ana Souza";           /* ok: sizeof(Nome) == 20      */
4  Nome a, b;
5  a = b;                           /* ERRO: vetor nao se atribui */
6
7  void registra(Nome x);           /* x chega como PONTEIRO, nao vetor */
```

## O apelido esconde a natureza do tipo

Quem lê `registra(Nome x)` pensa passar 20 bytes por cópia — mas em parâmetros, vetor vira endereço.

## Recomendação

Use typedef para struct, enum e escalares; para vetores, deixe o `[]` visível.

| Elemento                | Convenção usual  | Exemplo          |
|-------------------------|------------------|------------------|
| Apelido de tipo         | PascalCase       | Aluno, Situacao  |
| Constante de enumeração | MAIUSCULAS       | APROVADO         |
| Tag de struct/enum      | igual ao apelido | struct Aluno     |
| Variável e campo        | minúsculas       | media, matricula |

## Evite o sufixo `_t`

Nomes terminados em `_t` são reservados pelo POSIX (`size_t`, `time_t`). Criar `aluno_t` pode colidir com uma versão futura da biblioteca. Qualquer convenção serve — desde que seja a mesma do começo ao fim do projeto.

## O que aprendemos

- typedef dá um apelido a um tipo existente; não cria tipo novo.
- Leia qualquer typedef como uma declaração de variável com typedef na frente.
- Com struct/enum, elimina a repetição da palavra-chave.
- Com tipos escalares, documenta a intenção e concentra a mudança em uma linha.
- Com vetores, esconde comportamento — use com parcimônia.

**Registros que se explicam  
sozinhos**

```
1 typedef enum { MATUTINO, VESPERTINO,  
2             NOTURNO } Turno;  
3  
4 typedef enum { REPROVADO, RECUPERACAO,  
5             APROVADO, NUM_SITUACOES } Situacao;  
6  
7 typedef struct { int dia, mes, ano; } Data;  
8  
9 typedef struct {  
10     char    nome[20];  
11     int     matricula;  
12     Data    nascimento;  
13     double  media;  
14     Situacao situacao;  
15     Turno   turno;  
16 } Aluno;
```

## O que o tipo passou a dizer

- situacao tem três valores — e eles estão escritos.
- turno idem, com outros três.
- Nenhum int genérico sobrou valendo “0, 1 ou 2”.

## Lembre da aula passada

A ordem dos campos ainda decide o *padding*: Situacao e Turno ocupam 4 bytes, como int.

```
1  int cont[NUM_SITUACOES] = {0};
2
3  for (int i = 0; i < n; i++) {
4      turma[i].situacao = classifica(turma[i].media);
5      cont[turma[i].situacao]++;
6  }
7
8  for (Situacao s = REPROVADO; s < NUM_SITUACOES; s++) {
9      imprime_situacao(s);
10     printf(": %d aluno(s)\n", cont[s]);
11 }
```

## Duas leituras do mesmo `enum`

Como *valor* do campo e como *índice* do contador — o mesmo inteiro visto de dois ângulos. E o `for` percorre a enumeração inteira sem citar números.

## O que aprendemos

- `struct` agrupa campos; `enum` restringe e nomeia os valores de um campo; `typedef` dá nome ao conjunto.
- Campos com conjunto fechado de valores pedem `enum`, não `int`.
- A sentinela do `enum` dimensiona vetores de agregação.

## Teste de leitura

Se um colega consegue entender o que cada campo significa lendo só a declaração do `struct`, a modelagem está boa.

**Tarefas**

## Tarefa 1 — acrescentar um valor

Inclua `TRANCADO` na enumeração `Situacao`, *antes* da sentinela. Recompile com `-Wall` sem tocar em `imprime_situacao` e anote o que o compilador diz. Depois trate o novo caso.

## Tarefa 2 — mexer nos valores

Mude a declaração para `REPROVADO = 1`. Preveja o que acontece com `cont [NUM_SITUACOES]` *antes* de rodar; depois execute e explique o que observou.

## Tarefa 3 — apelidar tipos

Troque `int matricula` por `Matricula matricula`, com `typedef unsigned long Matricula`. Meça `sizeof (Aluno)` antes e depois e explique a diferença usando o que vimos sobre alinhamento.

## Relatório da turma

Escreva um programa que:

1. declare `Aluno turma[8]` com `Situacao` e `Turno`;
2. preencha os oito alunos no próprio código;
3. classifique cada aluno a partir da média, sem números mágicos;
4. imprima uma tabela com nome, matrícula, média, situação e turno;
5. imprima um histograma de asteriscos com a contagem por situação;
6. informe a média de cada turno, percorrendo a enumeração com um laço.

O tutorial traz o esqueleto, os testes e as perguntas de investigação.

enum nomeia um conjunto fechado de valores inteiros



Sem valores explícitos, a contagem começa em 0 na ordem escrita



switch sobre enum faz o compilador cobrar casos faltantes



typedef é apelido de tipo — não cria tipo novo nem checagem

## Próxima aula

Ponteiros: em vez de copiar o registro inteiro a cada chamada, passar o endereço — e o operador -> para chegar aos campos a partir dele.



André Backes.

*Linguagem C: completa e descomplicada.*

Elsevier, Rio de Janeiro, 2013.



Randal E. Bryant and David R. O'Hallaron.

*Computer Systems: A Programmer's Perspective.*

Pearson, Boston, 3 edition, 2015.

Seção 3.9: alinhamento de dados e layout de structs.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

*Algoritmos: teoria e prática.*

Elsevier, Rio de Janeiro, 3 edition, 2012.



Paul Deitel and Harvey Deitel.

*C: como programar.*

Pearson Prentice Hall, São Paulo, 6 edition, 2011.



Ulrich Drepper.

*What Every Programmer Should Know About Memory.*

Red Hat, Inc., 2007.

Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.



Ulrich Drepper.

How to write shared libraries.

Documento técnico, 2011.

Versão 4.1.2.



Paulo Feofiloff.

*Algoritmos em linguagem C.*

Elsevier, Rio de Janeiro, 2009.



ISO/IEC.

Iso/iec 9899:2018 — information technology: Programming languages: C.

Technical report, International Organization for Standardization, 2018.

Seção 6.7.2.1: structure and union specifiers.



Brian W. Kernighan and Dennis M. Ritchie.  
*C: a linguagem de programação — padrão ANSI.*  
Campus, Rio de Janeiro, 1989.



K. N. King.  
*C Programming: A Modern Approach.*  
W. W. Norton, New York, 2 edition, 2008.  
Capítulo 16: estruturas, uniões e enumerações.



Robert Mecklenburg.  
*Managing Projects with GNU Make.*  
O'Reilly Media, 3 edition, 2004.



Herbert Schildt.  
*C completo e total.*  
Makron Books, São Paulo, 3 edition, 1997.

-  Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.  
AddressSanitizer: A fast address sanity checker.  
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.  
Ferramenta usada no tutorial para detectar vazamentos e uso após free.
-  Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.  
*Estruturas de dados usando C*.  
Pearson Makron Books, São Paulo, 1995.
-  Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.  
Dynamic storage allocation: A survey and critical review.  
In *International Workshop on Memory Management (IWMM)*, pages 1–116. Springer, 1995.  
Panorama de estratégias de alocação dinâmica e fragmentação.
-  Nivio Ziviani.  
*Projeto de algoritmos: com implementações em Pascal e C*.  
Cengage Learning, São Paulo, 3 edition, 2011.